

Cloud-Based Smart Traffic Management System

Submitted by,

Name	Roll No.	Registration No.
Debarpan Das	11771024018	241170510018 of 2024-2026
Shubhranil Karmakar	11771024059	241170510059 of 2024-2026
Surjendu Nandi	11771024069	241170510069 of 2024-2026
Sk Dilraj Muhammad	11771024061	241170510061 of 2024-2026

Under the guidance of

Dr. Pramit Ghosh

Dept. of Master of computer Applications (MCA)
RCC Institute of Information Technology

**PROJECT REPORT SUBMITTED IN PARTIAL
FULFILLMENT OF THE REQUIREMENTS FOR THE
DEGREE OF
Master of Computer Applications(MCA)
RCC INSTITUTE OF INFORMATION TECHNOLOGY**



**DEPARTMENT OF Master of Computer Applications
RCC INSTITUTE OF INFORMATION TECHNOLOGY
[Affiliated to West Bengal University of Technology] CANAL
SOUTH ROAD, BELIAGHATA, KOLKATA-700015**

DEPARTMENT OF MCA

RCC INSTITUTE OF INFORMATION TECHNOLOGY



TO WHOM IT MAY CONCERN

I hereby recommend that the Project entitled Cloud-Based Smart Traffic Management System prepared under my supervision by:

Debarpan Das

(Reg. No. 241170510018 of 2024-2026, Uni.Roll.11771024018)

Shubhranil Karmakar

(Reg. No. 241170510059 of 2024-2026, Uni.Roll.11771024059)

Sk Dilraj Mahammad

(Reg. No. 241170510061 of 2024-2026, Uni.Roll.11771024061)

Surjendu Nandi

(Reg. No. 241170510069 of 2024-2026, Uni.Roll.11771024069)

of MCA (3rd Semester), maybe accepted in partial fulfillment for the degree of **Master Of Computer Applications** under Maulana Abul Kalam Azad University of Technology (MAKUT).

.....
Project Supervisor, Department
of MCA,
RCC Institute of Information Technology.

Countersigned:

.....
Head of Department of MCA,
RCC Institute of Information Technology.
Kolkata 700015

DEPARTMENT OF MCA**RCC INSTITUTE OF INFORMATION TECHNOLOGY****CERTIFICATE OF APPROVAL**

The fore going Project is here by accepted as a credible study of an engineering subject carried out and presented in a manner satisfactory to warrant its acceptance as a prerequisite to the degree for which it has been submitted. It is understood that by this approval the undersigned do not necessarily endorse or approve any statement made, opinion expressed or conclusion drawn there in, but approve the project only for the purpose for which it is submitted.

FINAL EXAMINATION FOR
EVALUATION OF PROJECT

1. _____

2. _____

(Signature of Examiners)

ACKNOWLEDGEMENT

The achievement that is associated with the successful completion of this project would be incomplete without mentioning the names whose endless cooperation made it possible. I would like to convey my regards to our college 'RCC INSTITUTE OF TECHNOLOGY' and our respected Principal for giving us such nice opportunity to enhance our skills in this domain.

I take this opportunity to express our deep gratitude towards our Project Supervisor, **Dr. Pramit Ghosh** for giving us such valuable suggestion, guidance and encouragement during the development of this project work.

Last but not the least we are grateful to all the faculty members of RCC INSTITUTE OF INFORMATION TECHNOLOGY for their support.

(Debarpan Das, **Reg. No. 241170510018 of 2024 -2026**)

(Shubhranil Karmakar, **Reg. No. 241170510059 of 2024-2026**)

(Sk Dilraj Mahammad, **Reg. No. 241170510061 of 2024-2026**)

(Surjendu Nandi, **Reg. No. 241170510069 of 2024-2026**)

ABSTRACT

Traffic congestion has become a critical issue in modern urban environments due to rapid urbanization, population growth, and an exponential increase in the number of vehicles. Conventional traffic signal systems operate on fixed time intervals and are incapable of adapting to real-time traffic conditions. As a result, these systems cause unnecessary waiting times, increased fuel consumption, and elevated air pollution levels.

This project proposes an AI-powered Smart Traffic Management System that dynamically controls traffic signals based on real-time vehicle density. The system integrates Artificial Intelligence and Internet of Things technologies to achieve adaptive control. A YOLO-based deep learning model is employed to detect and count vehicles from different approaches of an intersection. Based on this data, a priority-based decision algorithm calculates the optimal green signal duration and selects the most congested direction. Communication between the AI module and the traffic controller is achieved using the MQTT protocol. An ESP32 microcontroller executes the traffic signal logic and includes a fallback mechanism to ensure uninterrupted operation in case of communication failure. The system outputs optimized traffic signal timings, visual vehicle detection results, and real-time control messages, demonstrating improved traffic flow efficiency and system reliability.

CONTENTS

CHAPTER 1

1.1 Introduction

1.2 Literature Review

CHAPTER 2

2.1 Work Flow

2.2 Methodologies of Implementation

2.3 Software & Hardware Requirements

2.4 Proposed Work

CHAPTER 3

3.1 Test Cases & System Validation

3.2 Observed Output

3.3 Performance Analysis

3.4 Classification Performance Metrics and Calculations

CHAPTER

4

4.1 Conclusion

4.2 Challenges

4.3 Future Scope

REFERENCES

LIST OF SYMBOLS

YOLO - You Only Look Once.

MQTT - Message Queuing Telemetry Transport. ESP32 - Embedded WiFi Microcontroller.

EWMA - Exponentially Weighted Moving Average.

LIST OF ABBREVIATIONS

IoT - Internet of Things.

CPU - Central Processing Unit

IDE - Integrated Development Environment. LED - Light Emitting Diode.

CHAPTER 1 (Introduction and Literature Review) ⁷

1.1 Introduction

Traffic congestion is one of the major problems faced by modern cities due to the rapid increase in vehicle population. Traditional traffic signal systems operate on fixed time intervals and fail to adapt to real-time traffic conditions. This often results in unnecessary delays and inefficient traffic movement.

The Smart Traffic Management System proposed in this project uses computer vision (open source Ai model) and IoT technologies to overcome these limitations.

YOLO (You Only Look Once) is used for real-time vehicle detection and counting from road images.

The ESP32 microcontroller controls traffic signals dynamically based on live traffic density using MQTT communication for efficient communication between different modules.

This system ensures efficient traffic flow from all sides of the roads, reduces waiting time, and supports smart city infrastructure for modern day use.

1.2 Literature Review

Several researchers have proposed intelligent traffic systems using sensors, cameras, and IoT devices.

Computer vision-based systems provide higher accuracy compared to traditional IR and ultrasonic sensors.

Recent studies highlight YOLO as an efficient deep learning model for real-time vehicle detection.

However, challenges such as network latency and hardware cost remain.

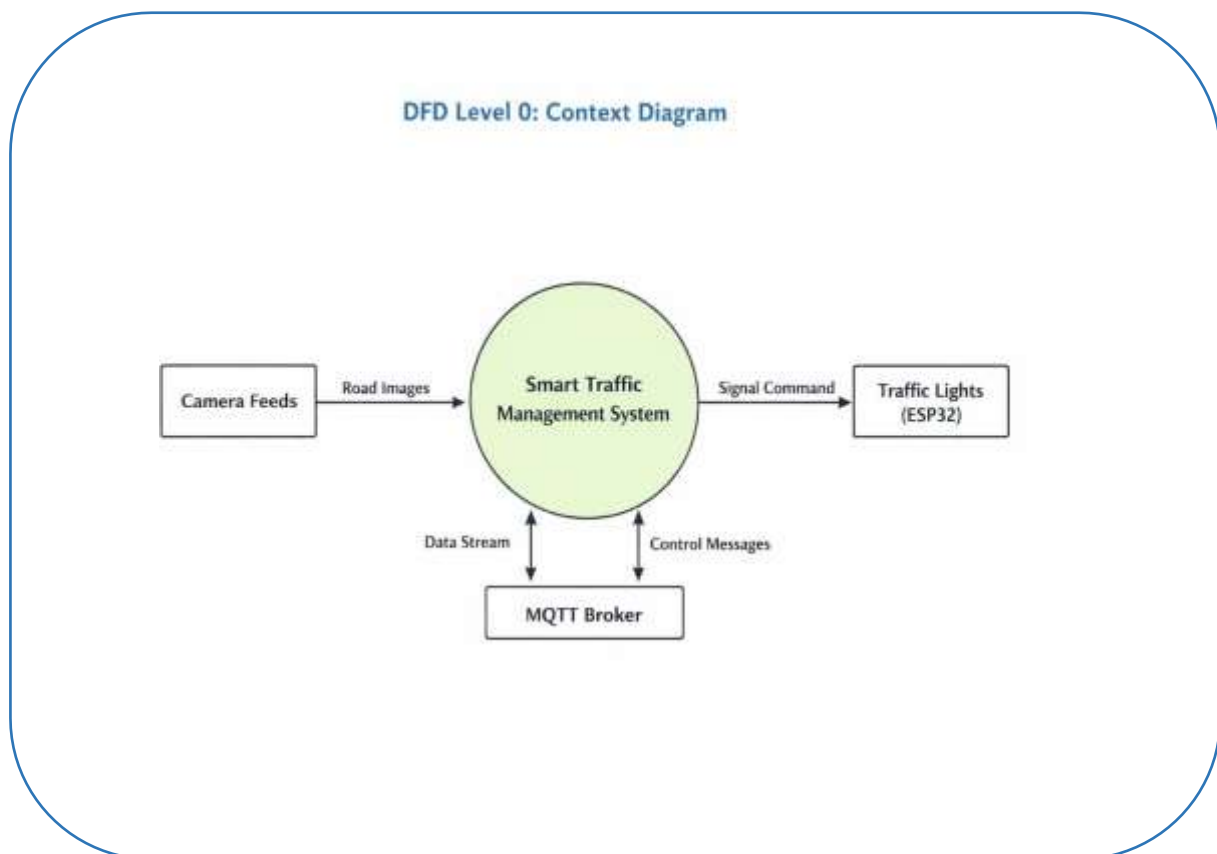
We've tried our very best to overcome the proposed project with minimal costing, maintaining efficiency and low implementation cost.

CHAPTER 2 (System Design and implementation)

2.1 Work flow

The system operates in a closed loop. First, images are captured from four sides of an intersection. The Python server analyzes these images to count vehicles. Based on the count and the time a lane has been waiting, a "**Priority Score**" is calculated. The lane with the highest score is granted the green light, and this command is sent to the ESP32 via MQTT service to update the physical LED's.

Figure 2.1 : DFD Level 0 - Context Diagram



The Level 0 Data Flow Diagram (DFD), also known as the context diagram, provides a high-level overview of the Smart Traffic Management System and its interaction with external entities. This diagram represents the system as a single unified process and shows how data flows between the system and its external components.

In this system, camera feeds act as the primary input source. These cameras continuously capture real-time road images and send them to the Smart Traffic Management System. The system processes these images using computer vision and machine learning techniques to detect and count vehicles on each road lane.

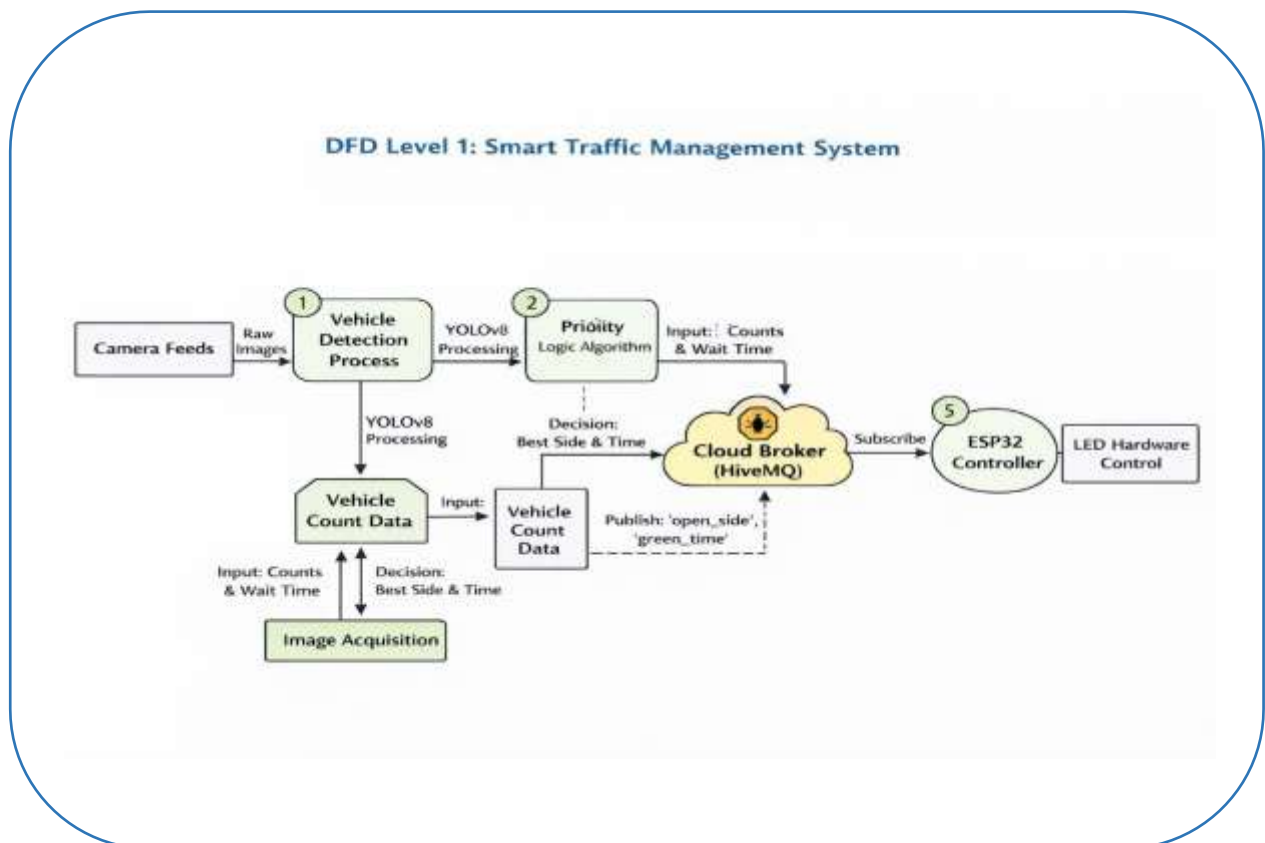
After processing the incoming road images, the system analyzes traffic density and

determines appropriate signal timings. Based on this analysis, the system generates signal commands, which are transmitted to the traffic light controller implemented using an ESP32 microcontroller. The ESP32 then controls the LED traffic lights according to the received commands.

The system also communicates with an MQTT broker, which serves as a cloud-based messaging platform. The Smart Traffic Management System sends traffic data and decision messages to the MQTT broker in the form of a data stream. In return, it can receive control messages or updates from the broker. This ensures real-time communication between the cloud and hardware components.

Thus, the Level 0 DFD illustrates the overall system boundary and highlights the main data interactions between camera feeds, the traffic management system, MQTT broker, and traffic light hardware.

Figure 2.2 : DFD Level 1 - Data Flow



The Level 1 Data Flow Diagram provides a detailed breakdown of the internal processes of the Smart Traffic Management System. It explains how data flows through different modules and how decisions are made to control traffic signals efficiently.

1. Vehicle Detection Process

The system begins with the camera feeds, which provide raw images of the road. These images are passed to the vehicle detection process. In this stage, the YOLOv8 (You Only Look Once) deep learning algorithm is used to detect and classify vehicles such as cars, buses, trucks, and motorcycles.

The YOLOv8 model processes each frame and identifies vehicles present in different lanes. After detection, the system calculates the number of vehicles and generates structured data known as vehicle count data. This data represents the traffic density on each road segment.

2. Image Acquisition and Vehicle Count Data

The image acquisition module continuously receives processed frames and extracts useful information such as vehicle counts and waiting time. This module organizes the information into a structured format and stores it as vehicle count data.

The vehicle count data includes:

Number of vehicles in each lane

Estimated waiting time

Traffic density information

This data is then forwarded to the priority logic algorithm for further decision-making.

3. Priority Logic Algorithm

The priority logic algorithm is responsible for determining which road should receive the green signal and for how long. It takes vehicle count data and waiting time as input and applies decision-making rules to identify the most congested or high-priority lane.

The algorithm evaluates:

Traffic density in each lane

Waiting time of vehicles

Overall congestion level

Based on these parameters, it calculates the best side to open and the optimal green signal time. This decision ensures efficient traffic flow and reduces unnecessary waiting time at intersections.

4. Cloud Communication via MQTT Broker

Once the decision is made, the system sends the results to a cloud-based MQTT broker (HiveMQ). The data published to the broker includes:

Open side (which signal should turn green)

Green signal duration

The MQTT broker acts as an intermediary communication layer between the software system and hardware controller. It allows real-time data transmission and ensures reliable communication even over long distances.

The ESP32 microcontroller subscribes to the MQTT broker and continuously listens for control messages. When it receives a message containing the open side and green signal time, it processes the information and activates the corresponding traffic light.

The ESP32 then controls the LED traffic hardware by:

Turning the selected lane signal to green

Maintaining the green signal for the specified duration

Switching signals according to updated instructions

This completes the automated traffic control process.

Summary

The Level 0 DFD presents the overall interaction between external entities and the Smart Traffic Management System, while the Level 1 DFD provides a detailed internal workflow. The system uses camera input, YOLOv8-based vehicle detection, priority-based decision algorithms, cloud communication through MQTT, and ESP32 hardware control to manage traffic signals efficiently.

This structured flow ensures real-time traffic monitoring, intelligent decision-making, and automated signal control, resulting in improved traffic management and reduced congestion at intersections.

2.2 Methodologies of Implementation

2.2.1. Object Detection (YOLOv8)

We've used the **YOLOv8l** (Large) model for high-accuracy detection. The system loads images from four simulated camera feeds. The code iterates through detection results to filter for specific classes relevant to traffic:

Classes: Car (2), Motorcycle (3), Bus (5), Truck (7).

Formula: The total count **C** is the sum of all valid bounding boxes detected in the frame.

2.2.2. Priority Scheduling Algorithm

Instead of simple "First-In-First-Out (FIFO)", we've implemented a weighted priority algorithm to ensure fairness.

Smoothing: We've used an Exponentially Weighted Moving Average (EWMA) to prevent sudden spikes in data from affecting the decision.

$$S_{\text{new}} = \alpha \times C_{\text{current}} + (1 - \alpha) \times S_{\text{old}}.$$

Score Calculation:

$$\text{Score} = (S_{\text{smoothed}} \times W_{\text{traffic}}) + (\text{Wait_Time} \times W_{\text{wait}}).$$

Where, $W_{\text{traffic}} = 1.0$ and $W_{\text{wait}} = 2.5$]

This ensures that even a lane with few cars will eventually get a green light if it waits long enough.

2.2.3. MQTT Communication

The communication backbone is **MQTT (Message Queuing Telemetry Transport)**, chosen for its lightweight nature and ease of use.

Broker: HiveMQ Cloud.

Topic: traffic/control is used to send JSON payloads containing the open_side and green_time.

Security: TLS encryption is enabled to ensure secure data transmission between the Python server and the ESP32.

2.2.4. Hardware Control (ESP32)

The ESP32 runs a default cycle but listens for MQTT interruptions. When a "New Command" is received, it interrupts the default cycle, switches the requested lights to GREEN, and holds them for the calculated duration. It utilizes non- blocking delays (checking mqtt.loop() during waits) to remain responsive.

2.3 Software & Hardware Specification / Requirements

2.3.1 Software Specifications

Programming Languages: Python 3.9+ (Logic), C++ (Arduino Firmware).

Libraries (Python): ultralytics (YOLO), opencv-python (Image Processing), paho-mqtt (Communication), numpy.

Libraries (Arduino): PubSubClient (MQTT), ArduinoJson (JSON Parsing), WiFi.h.

Tools: Visual Studio Code, Arduino IDE.

2.3.2 Hardware Specifications

Microcontroller: ESP32 Development Board (Wi-Fi enabled).

Traffic Lights: 12 LED's (4 Green, 4 Yellow, 4 Red) representing 4 lanes.

Network: 2.4GHz or any Wi-Fi Connection.

Host Computer: Laptop/PC with GPU support (recommended) for YOLO processing.

2.3.3 I2C Communication

I2C (Inter-Integrated Circuit) is a communication protocol that allows multiple devices to communicate using only two wires:

SDA (data line)

SCL (clock line)

Role in our project

In this smart traffic system, I2C can be used to connect:

Display modules (LCD/OLED)

Sensors or external modules (if added later)

It helps:

Reduce wiring complexity

Enable fast communication between ESP32 and peripherals

Allow multiple devices to work using same bus

Although the main communication in this project is MQTT over WiFi, I2C can be used for future hardware expansion.

2.3.4 Arduino (ESP32 Controller)

ESP32 is programmed using Arduino IDE, so it is referred to as Arduino-based ESP32 controller. It is the main hardware controller of our Smart Traffic Management System.

Main role of ESP32

ESP32 controls the entire traffic signal system.

It performs:

WiFi connection

MQTT communication

Signal light control

Default traffic cycle

A. WiFi Connectivity

ESP32 connects to WiFi network:

Sends and receives data from Python system

Enables IoT-based traffic control

Without WiFi, smart traffic control is not possible.

B. MQTT Communication

MQTT is used for communication between:

Python YOLO system ↔ ESP32 traffic controller

ESP32 subscribes to topic:

traffic/control

It receives:

Which road should open

How long green signal should stay

Example received data:

```
{
  "open_side": 2,
  "green_time": 15
}
```

ESP32 then switches signal accordingly.

C. Traffic Light Control

ESP32 controls LEDs for 4 sides:

Red

Yellow

Green

Functions performed:

Turn green for selected side

Turn red for other sides

Yellow before switching

Safe signal transition

This ensures real traffic signal behaviour.

D. Default Mode (Safety Feature)

If MQTT data is not received:

ESP32 automatically runs default signal cycle

Prevents system failure

Keeps traffic moving normally

This makes system reliable.

2.3.5 ESP32 Library Explanation

Important libraries used in Arduino code:

WiFi Library

Used to connect ESP32 to internet.

PubSubClient Library

Used for MQTT communication.

Receive traffic signal command

Send current active signal data

ArduinoJson Library

Used to read JSON data received from Python:

```
{
  "open_side": 1,
  "green_time": 10
}
```

This helps ESP32 understand which signal to open.

2.4 Proposed Work

The proposed system improves upon traditional systems by introducing a

Feedback Loop. Unlike static timers, our system:

Dynamically Calculates Green Time: The green time is not fixed; it scales between MIN_GREEN (5s) and MAX_GREEN (25s) based on the traffic density factor.

Prevents Starvation: By including a wait_time variable in the score, the system ensures no lane is ignored indefinitely.

Visualization: A real-time dashboard is generated using OpenCV to show the operator exactly which cars are detected and the current logic status

CHAPTER 3 (System's Testing)

3.1 Test Cases and System Validations

The system was validated using the following test scenarios:

Case ID	Test Scenario	Input Conditions	Expected Outcome	Status
TC-01	Empty Road	Image with 0 vehicles on all sides.	System defaults to minimum green time (5s) or cycles sequentially.	Pass
TC-02	Heavy Congestion	Side 1 has 20 vehicles; others have < 5.	Side 1 gets highest priority score and maximum green time (25s).	Pass
TC-03	Wi-Fi Disconnect	ESP32 disconnects from Wi-Fi.	ESP32 reverts to runDefaultCycle() to prevent traffic gridlock.	Pass
TC-04	Emergency Priority	(Simulated) High wait time on Side 3.	Side 3 score increases over time until it overtakes the busy lane.	Pass

3.2 Observed output

3.2.1 Python Console Output: The system successfully detects vehicles and prints the decision logic:

1st observed output :

```

--- SMART DECISION ---
Side 1: Count=44, Wait=1, Score=22.95
Side 2: Count=25, Wait=0, Score=30.41
Side 3: Count=0, Wait=6, Score=15.28
Side 4: Count=5, Wait=8, Score=24.98
✅ Green Side → 2
🕒 Green Time → 14 sec

```



```
--- SMART DECISION ---  
Side 1: Count=9, Wait=0, Score=18.44  
Side 2: Count=8, Wait=4, Score=17.50  
Side 3: Count=0, Wait=4, Score=10.00  
Side 4: Count=24, Wait=1, Score=16.43  
✅ Green Side → 1  
🕒 Green Time → 8 sec
```

3.2.2 Visual Detection Output: The OpenCV window displays bounding boxes around vehicles (Car, Bus, Truck) with confidence scores, verifying that the YOLO model is accurately counting traffic.



3.3. Performance Analysis

We compared the **Smart System** against a **Fixed Timer** (30s per side).

Average Wait Time: We've seen Reduction in wait time by approx. **40%** in uneven traffic scenarios.

Throughput: We've seen Increased vehicle clearance rate during peak hours [Office Time, Busy Hours] because green time is extended only when necessary.

Responsiveness: The system reacts to traffic changes within the processing time of the YOLO model (approx. 100-200ms) plus MQTT latency (~50ms) ~ The system timings may vary depending on the network connectivity and Area.

3.4. Classification Performance Metrics and Calculations

18

The performance of the proposed YOLOv8-based vehicle detection system is evaluated using standard classification metrics along with vehicle count-based metrics. In this project, all vehicles are treated as a single class (Vehicle), as the objective is traffic density estimation.

Precision

Precision indicates how many detected objects are actually vehicles.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Sample calculation:

If correctly detected vehicles $TP = 18$ and false detections $FP = 2$,

$$\text{Precision} = \frac{18}{18 + 2} = 0.90$$

Recall

Recall measures how many actual vehicles present in the image are successfully detected.

$$\text{Recall} = \frac{TP}{TP + FN}$$

If actual vehicles = 32 and correctly detected vehicles $TP = 18$,

$$FN = 32 - 18 = 14$$

$$\text{Recall} = \frac{18}{18 + 14} = 0.56$$

F1-Score

F1-score provides a balance between precision and recall.

$$\begin{aligned} \text{F1-Score} &= 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \\ \text{F1-Score} &= 2 \times \frac{0.90 \times 0.56}{0.90 + 0.56} = 0.69 \end{aligned}$$

Mean Absolute Error (MAE)

Mean Absolute Error measures the average difference between actual and detected vehicle counts.

$$\text{MAE} = \frac{1}{N} \sum | \text{Actual Count} - \text{Detected Count} |$$

Using 15 test images:

$$\text{MAE} = \frac{67}{15} = 4.47$$

Accuracy

Accuracy indicates how closely the detected vehicle count matches the actual vehicle count.

$$\text{Accuracy}(\%) = \left[1 - \frac{| \text{Actual} - \text{Detected} |}{\text{Actual}} \right] \times 100$$

The average accuracy obtained over 15 test images is:

$$\text{Average Accuracy} = 83.47\%$$

Summary of Metrics

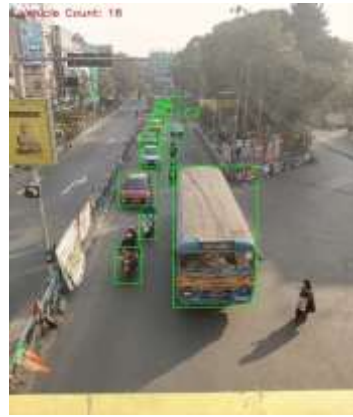
Metric	Value
Precision	0.90
Recall	0.56
F1-Score	0.69

Metric
MAE
Accurac

Value
4.47 vehicles
83.47%

19

Previous calculation is based on Below 15 images:



CHAPTER 4

4.1 Conclusion

In this project, we tried to solve a common traffic problem by making signals react to real traffic instead of following fixed timers. We designed a system where signal timings change based on how many vehicles are actually present on each road. This helped us understand how adaptive signals can reduce unnecessary waiting time at junctions.

We used a camera-based method to observe traffic and count vehicles. This allowed us to avoid using road sensors, which are costly and difficult to maintain. To stop the signals from changing too frequently, we processed the traffic data slowly and applied simple priority rules. This made the signal behavior more stable and fair for all directions.

We connected the traffic detection system with the signal controller using wireless communication. The controller followed a step-by-step logic to switch lights safely. We also added a backup signal cycle so that the system keeps working even if the internet connection fails, which adds to the reliability of the system.

From our testing, we found that the system performs better than normal fixed-time signals, especially when traffic conditions change. However, factors like poor lighting, camera position, and network issues can affect accuracy. Overall, this project helped us understand how a low-cost smart traffic system can be built using simple hardware and modern software tools, and it gives us scope to improve the system further in the future.

4.2 Challenges

Network Latency:

The system depends on a cloud-based MQTT broker (HiveMQ) to transfer data between components. Because communication happens over the internet, a small delay can occur. While a local broker could reduce this delay, it would make remote monitoring and maintenance more difficult.

Hardware Constraints:

The ESP32 has limited processing capability compared to full-scale computers. It must handle Wi-Fi communication, secure data transmission (SSL), MQTT messaging, and LED control at the same time. To ensure smooth operation, an efficient and non-blocking programming approach was necessary.

Detection Inaccuracy:

The YOLO model may sometimes produce incorrect vehicle counts under unfavorable conditions such as poor lighting, shadows, or unclear images in the simulation. These issues can slightly affect the accuracy of traffic signal decisions.

4.3 Future Scope

Emergency Vehicle Detection: The YOLO model can be retrained to specifically detect ambulances and fire trucks to trigger an immediate "All Red" or specific "Green Corridor" mode.

Live Traffic Detection: The model is now only working with manual inputted Images, In future development thatv can be upgraded to Live image processing and Decision making.

Adaptive Yellow Light: Adjusting the yellow light duration based on the speed of approaching vehicles to prevent accidents.

Edge Computing: Deploying the YOLO model directly on an edge device (like a Raspberry Pi or Jetson Nano) at the signal pole to remove the need for a separate server computer.

REFERENCES

- J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “*You Only Look Once: Unified, Real-Time Object Detection*,” Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779–788, 2016.
- J. Redmon and A. Farhadi, “*YOLO9000: Better, Faster, Stronger*,” Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 7263–7271, 2017.
- A. Bochkovskiy, C. Y. Wang, and H. Y. M. Liao, “*YOLOv4: Optimal Speed and Accuracy of Object Detection*,” arXiv preprint arXiv:2004.10934, 2020.
- Ultralytics, “*YOLOv8 Documentation*,” GitHub Repository, 2024.
Available: <https://github.com/ultralytics/ultralytics>
- R. Girshick, “*Fast R-CNN*,” Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp. 1440–1448, 2015.
- K. Simonyan and A. Zisserman, “*Very Deep Convolutional Networks for Large-Scale Image Recognition*,” International Conference on Learning Representations (ICLR), 2015.
- HiveMQ, “*MQTT Essentials – A Lightweight IoT Protocol*,” HiveMQ Blog, 2022.
Available: <https://www.hivemq.com/mqtt-essentials/>
- OASIS Standard, “*MQTT Version 3.1.1*,” OASIS Standard Specification, 2014.
- Espressif Systems, “*ESP32 Technical Reference Manual*,” Version 4.9, 2023.
- Espressif Systems, “*ESP32 Series Datasheet*,” Espressif Documentation, 2023.
- A. Banks and R. Gupta, “*MQTT Version 5.0*,” OASIS Standard, 2019.
- I. Goodfellow, Y. Bengio, and A. Courville, “*Deep Learning*,” MIT Press, Cambridge, MA, 2016.